

Design Model, Design Classes & Component-Level Design

Two things trimmed from this topic per your call: (1) the component-level "basic design principles" — OCP, LSP, DIP, ISP — since these are the same SOLID principles already covered in full depth with code examples in the OOPS [09-solid-principles.md](#) draft; (2) a re-explanation of coupling/cohesion, already covered in the Design Concepts topic. What remains is the material that's genuinely new: design class types, the design model's structure, packaging principles, and the component-level design process.

Part 1: From Analysis Classes to Design Classes

As development moves from analysis to design, the **analysis classes** identified in requirements engineering (see the Requirements Engineering topic) are refined into **design classes** — design classes add the technical detail (data structures, method definitions, interactions) needed to actually implement and execute the system. As the design model evolves: abstraction level decreases, business-oriented descriptions get replaced by implementation-oriented detail, and design classes end up specifying exact methods, attributes, interfaces, and collaborations. **Design classes are, in effect, the blueprint handed to the coder.**

1. The Five Types of Design Classes

Type	Responsibility	Visible to end user?
User Interface Classes	All abstractions needed for human-computer interaction — screens, windows, forms, dialogs, user input/output	Yes — this is what the user sees
Business Domain Classes	Refinements of analysis classes; represent real-world business entities, define attributes + services, implement core business rules	Indirectly — the heart of the application's logic
Process Classes	Lower-level business logic that <i>supports</i> domain classes — coordinates workflows and complex operations spanning multiple business objects; control logic, not data storage	No
Persistent Classes	Represent data that survives beyond program execution — manage databases/files, provide save/retrieve/update/delete, keep persistence logic separate from business logic	No
System Classes	Software management/control functions — OS and network communication, memory management, error handling, security, resource allocation	No

When would you actually need a Process class rather than just putting the logic in a Business Domain class? When a business rule is too complex to fit cleanly inside a single domain class — e.g., "approve a loan" might require coordinating a `Customer`, a `CreditCheck`, and a `LoanPolicy` object together; rather than cramming that orchestration logic into one of those three domain classes (making it responsible for far more than its own data), a dedicated `LoanApprovalProcess` class handles the coordination, keeping each domain class focused only on its own data and rules.

2. Well-Formed Design Classes (Arlow & Neustadt)

Every design class should satisfy four characteristics to be considered well-formed:

1. **Complete and Sufficient** — includes *all* attributes and methods logically associated with its purpose (complete), and *only* those, with no redundant or unrelated behavior (sufficient). This avoids bloated, unfocused classes.
 2. **Primitiveness** — each method performs one specific service, and provides only *one* way to perform that service. Avoids duplicate functionality and simplifies maintenance (no ambiguity about "which method do I call for this?").
 3. **High Cohesion** — a small, focused set of responsibilities; methods operate on the same related data. Improves readability, reliability, and reusability.
 4. **Low Coupling** — collaborates with other classes only when necessary, with minimal dependency. This is reinforced by the **Law of Demeter**: a method should communicate only with closely related ("neighbor") classes — not reach through one object to grab and manipulate a second object several levels removed. (Colloquially: "don't talk to strangers," or "only talk to your immediate friends.")
-

Part 2: The Design Model

3. Two Dimensions of the Design Model

The design model — the overall blueprint for building software, analogous to an architect's plan for a house — can be viewed along two dimensions:

1. **Process Dimension** — how the design evolves over time, as a sequence of design activities (architectural design → interface design → component design → deployment design), not as one single step.
2. **Abstraction Dimension** — the level of detail, starting high (close to the problem domain) and refining toward implementation-level detail. The boundary between analysis and design is sometimes sharp, sometimes gradual — analysis models often blend smoothly into design models rather than switching over all at once.

The design model reuses many of the **same UML diagrams** as the analysis model, but design-stage diagrams are more detailed, add implementation-specific information, and emphasize architecture, components, interfaces, and interactions rather than just problem-domain concepts.

Design elements aren't created strictly sequentially: architectural design is usually done first, interface design and component-level design often happen in parallel, and deployment design is usually finalized once the others have stabilized. Design patterns can be applied at any stage.

4. Ten Design Modeling Principles

#	Principle	Core idea
1	Traceability to Requirements	Every design element must trace back to the requirements model
2	Architecture First	Architecture is the skeleton — component design follows it, not the reverse
3	Data Design Is Critical	Poor data design → inefficient processing, tangled logic
4	Careful Interface Design	Good interfaces reduce errors, ease integration and testing
5	User Interface Focus	A poor UI can make even a well-designed system feel unusable
6	Functional Independence	Achieved through high cohesion (<i>see Design Concepts topic</i>)
7	Low Coupling	Reduces error propagation, improves maintainability (<i>see Design Concepts topic</i>)
8	Understandable Design	Must be clear to developers, testers, <i>and</i> maintainers, not just its author
9	Iterative Design	Design evolves across multiple passes, each one simplifying/improving it
10	Design Supports Agile Development	Design documentation still matters in agile — it should stay synchronized with the code, not go stale

Why "Architecture First" specifically? Because architecture is the one design decision that's hardest and most expensive to change *after the fact* — every other design element (interfaces, components, data structures) is built assuming a particular architectural shape, so getting the architecture right (or at least directionally right) before investing heavily in component-level detail avoids having to redo that downstream detail when the architecture eventually turns out to be wrong. This is the same "architecture gives the highest ROI" point from the Architectural Styles topic, restated as a design-process rule.

5. Design Model Elements (the Deliverables of Design)

The requirements model — its scenario-based elements (use cases, user stories), class-based elements (analysis classes, CRC models), and behavioral elements (state/sequence diagrams) — feeds into design and is systematically transformed into five categories of design deliverables:

Element	What it defines	Derived from	Represented via
Data / Class Design	Design-class realizations of analysis classes; how data is organized, stored, accessed	Analysis classes, CRC models	Class diagrams
Architectural Design	Overall system structure — relationships between major elements, the chosen architectural style, applied design patterns, constraints (performance/scalability/security)	Application domain knowledge, the requirements model, architectural styles/patterns	Architecture/component diagrams
Interface Design	How the software interacts with users, external systems, and its own internal components; must include error handling and security	Usage scenarios and behavioral models	UI mockups, interface specs
Component-Level Design	Procedural description of each component's internal logic — local data structures, algorithms, interfaces to its own services	Class-based + behavioral models, the architectural design	Component diagrams, activity diagrams, pseudocode
Deployment-Level Design	How software components are deployed onto physical hardware nodes	The architectural design	UML deployment diagrams (descriptor form = general hardware, or instance form = specific hardware configuration)

Data/Class design and Architectural design may proceed partly in parallel, with detailed class design finalized later during Component-Level design; Deployment design is typically finalized last, once the others have stabilized — matching the process-dimension ordering from Section 3.

6. Why Software Design Matters

The importance of software design can be summarized in one word: **QUALITY**.

- It's the **only** phase where quality can be systematically *built in* — every later phase can only preserve or fail to preserve quality that design already established, not create it from nothing.
- It provides representations that can be evaluated *before* any code is written — catching structural problems while they're still cheap to fix.
- It's what accurately translates stakeholder requirements into a working system, and forms the foundation everything downstream (coding, testing, maintenance) is built on.

Without proper design: systems become unstable, small changes trigger disproportionately large failures, testing becomes difficult (there's no clear internal structure to test against), and quality issues surface late — precisely when the least time and budget remain to fix them. This is the same cost-of-change argument from the Requirements Engineering & Agile topic, applied specifically to *design* defects rather than requirements defects.

Part 3: Component-Level Design

7. What Component-Level Design Does

Component-level design uses information from both the requirements model and the architectural model. In object-oriented software engineering, it focuses on refining problem-domain classes, defining supporting infrastructure classes, and specifying exact attributes, operations, and interfaces — this detailed design is the direct precursor to coding.

8. Packaging Principles (Component Organization)

Components are usually grouped into packages or subsystems. Three principles guide how that grouping should be done:

1. **Reuse/Release Equivalency Principle (REP)** — the unit of reuse should be the unit of release. Components meant to be reused together should be packaged together and released/versioned together — you can't sensibly reuse "half a package" if it's only ever released as a whole.
2. **Common Closure Principle (CCP)** — classes that **change together** should be packaged together. This minimizes the blast radius of a modification: if a change to one business rule requires editing three classes, and all three live in the same package, the impact of that change is contained to one package rather than scattered across several.
3. **Common Reuse Principle (CRP)** — classes that are **not** reused together should **not** be grouped together. If package X bundles class A (used everywhere) with class B (used only in one obscure feature), then every consumer of A is forced to also depend on, recompile, and re-test against B —

even though they never actually use it. CRP avoids this unnecessary coupling of unrelated reuse needs.

REP vs. CCP vs. CRP in one line: REP is about what you *release* together, CCP is about what tends to *change* together, and CRP is about what actually gets *reused* together — a good package satisfies all three simultaneously, but they can pull in different directions (e.g., grouping by "changes together" might group classes that aren't actually reused together), which is why packaging is a genuine design decision, not a mechanical one.

9. Component-Level Design Guidelines

For components:

- Use clear naming conventions. Architectural (problem-domain) component names should be meaningful to stakeholders; infrastructure/implementation components should reflect their technical purpose.
- Use **stereotypes** to classify components, e.g., `<<infrastructure>>`, `<<database>>`, `<<table>>`.

For interfaces:

- Interfaces describe communication between components and directly support the Open-Closed Principle (from OOPS SOLID) — new components can plug into an existing interface without modifying the components already using it.
- To reduce diagram clutter, use **lollipop notation** (a small circle/line, rather than a full UML interface box) once diagrams get complex.
- Model interfaces flowing consistently from the left-hand side of a component, and show only the interfaces relevant to the current discussion, even if a component technically exposes others.

For dependencies and inheritance:

- Model dependencies left to right.
- Model inheritance bottom (derived class) to top (base class).
- Prefer interface-based dependencies over direct component-to-component dependencies — this improves maintainability and, again, aligns with the Open-Closed Principle.

10. The Component-Level Design Process (7 Steps)

1. Identify design classes — problem domain
 2. Identify design classes — infrastructure domain
 3. Elaborate all non-reusable design classes
 4. Describe persistent data sources
 5. Develop behavioral representations
 6. Elaborate deployment diagrams
 7. Refactor and evaluate alternatives
- ↑ _____ ↓ (iterative — loops back)

1. **Identify design classes in the problem domain** — derived from the analysis classes and architectural components already established; each is elaborated with enough implementation-level detail to support coding directly.

2. **Identify design classes in the infrastructure domain** — classes that support the system but aren't part of the problem domain and typically don't appear in the requirements model at all (GUI components, OS interfaces, data/object management classes).
 3. **Elaborate all non-reusable design classes** — for every class not being acquired as an existing reusable component, fully describe its interfaces, attributes, and operations in implementation-ready detail, applying high cohesion/low coupling heuristics along the way.
 4. **Describe persistent data sources** — identify and describe databases/files, refine their structure, and define the classes responsible for managing that persistent data.
 5. **Develop behavioral representations** — UML state diagrams (see the UML Modeling Diagrams topic) describe how each design class behaves over time, in response to events.
 6. **Elaborate deployment diagrams** — refine them with implementation-level detail: which hardware/OS environments exist, and where major component packages get deployed.
 7. **Refactor and evaluate alternatives** — component-level design is inherently iterative; representations are reviewed and refactored for consistency, completeness, and accuracy, and alternative solutions are weighed using the established design principles before settling on the final version.
-

Interview Questions With Answers

Q1. What's the practical difference between a Business Domain class and a Process class, and why can't process logic just live inside the relevant domain class?

Answer: A Business Domain class represents a real-world business entity and holds the attributes and behavior directly belonging to that entity — for instance, a `LoanApplication` class holding its own data and simple validation rules. A Process class exists specifically to coordinate *workflows that span multiple* business domain objects — control logic rather than data storage — and is used when a business rule is too complex to sensibly fit inside any single domain class without that class taking on responsibilities well beyond its own data. Putting process logic directly into a domain class would violate the "complete and sufficient" and high-cohesion criteria for well-formed design classes, since the domain class would end up containing behavior unrelated to its own core data (e.g., a `Customer` class that also orchestrates credit checks, policy lookups, and approval workflows involving several *other* objects) — separating that coordination logic into a dedicated Process class keeps each domain class focused on just its own responsibilities.

Q2. Explain the Law of Demeter and how it relates to the "low coupling" criterion for well-formed design classes.

Answer: The Law of Demeter states that a method should only communicate with its closely related, "neighbor" objects — it shouldn't reach through one object to access and manipulate a second object that object happens to hold a reference to (colloquially, "don't talk to strangers"). This directly reinforces low coupling because a method that reaches deep into an unrelated object's internal structure

(e.g., `customer.getAccount().getBank().getAddress().getZipCode()`) creates a hidden dependency on the internal structure of *every* object in that chain, not just the one it's supposed to be collaborating with directly — so if any intermediate object's internal structure changes, this method breaks, even though it has no direct, obvious relationship to that intermediate object. Following the Law of Demeter keeps each class's dependencies limited to the objects it's meant to directly collaborate with, which is exactly what "low coupling" is trying to achieve at the class level.

Q3. Why is "Architecture First" listed as a specific design modeling principle rather than assuming design naturally proceeds architecture-then-details anyway?

Answer: It's called out explicitly because there's a real, common failure mode where teams under time pressure jump straight into writing component- or class-level code without deliberately establishing the architecture first, reasoning that "the architecture will emerge" from enough individually well-designed components — but architecture isn't just the sum of good component decisions, since it governs cross-cutting concerns (how components communicate, what the overall structural style is, what constraints apply system-wide) that no single component-level decision can establish on its own. Naming "Architecture First" as a principle is a corrective against that failure mode: it insists that architectural decisions be made deliberately and early, precisely because architecture is the hardest and most expensive layer to change retroactively once a large amount of component-level work has already been built on top of an implicit, accidental architecture that nobody consciously chose.

Q4. Give an example where the Common Closure Principle (CCP) and the Common Reuse Principle (CRP) might pull a packaging decision in opposite directions.

Answer: Suppose an `Invoice` class and a `TaxCalculator` class tend to change together whenever tax law changes (satisfying CCP, which would suggest packaging them together), but `TaxCalculator` is also independently reused by an entirely separate `PayrollReport` feature that has nothing to do with invoicing and never uses `Invoice` at all (which CRP would flag as a reason *not* to bundle `TaxCalculator` with `Invoice`, since `PayrollReport` would then be forced to depend on, and be affected by changes to, the unrelated `Invoice` class it never actually uses). This is a genuine tension: CCP argues for grouping by what tends to change together, while CRP argues for grouping by what's actually reused together, and here those two criteria disagree about the right home for `TaxCalculator`. Resolving it usually means factoring `TaxCalculator` into its own package, reused independently by both `Invoice`-related code and `PayrollReport`-related code, rather than fully satisfying either CCP or CRP at the expense of the other.

Q5. What is lollipop notation, and why would a design team switch to it rather than continuing to use full UML interface boxes as a diagram grows?

Answer: Lollipop notation represents an interface as a small circle (or a circle-and-line, resembling a lollipop) attached to the component that provides it, rather than drawing out a full, separate UML interface box with its complete list of method signatures. A team switches to it as a diagram grows specifically to manage visual complexity — a component diagram showing many components, each exposing several interfaces, becomes cluttered and hard to read quickly if every interface is drawn as a

full box with its own compartments; lollipop notation keeps the *existence* and *provider* of each interface visible at a glance without forcing the reader to parse full interface definitions directly on the diagram, trading some detail (which has to be looked up elsewhere, e.g., in an interface specification) for overall diagram readability.

Q6. Why does the component-level design process identify infrastructure-domain design classes as a separate step from problem-domain design classes, rather than treating all design classes the same way?

Answer: Problem-domain design classes are direct refinements of the analysis classes already identified during requirements engineering — they represent the actual business entities and logic the system exists to handle, and there's already a requirements-model trail connecting them back to stakeholder needs. Infrastructure-domain classes (GUI components, OS interfaces, data/object management classes) typically don't appear in the requirements model at all, and may not even be anticipated in the architectural model — they exist to support the *implementation* of the system rather than to represent anything a stakeholder asked for directly. Treating them as a separate identification step matters because they need to be actively discovered and specified from scratch during design (there's no earlier artifact to refine them from), whereas problem-domain classes are being progressively refined from an already-existing analysis-class starting point — conflating the two steps risks either under-specifying necessary infrastructure classes (because nobody explicitly went looking for them) or wasting refinement effort by treating already-analyzed problem-domain classes as if they needed to be discovered from nothing.

Q7. Why is step 7 of the component-level design process ("refactor and evaluate alternatives") placed last, and why is the process described as iterative rather than linear despite being numbered 1 through 7?

Answer: Step 7 is placed last in the *listing* because it depends on having concrete design classes, persistent data descriptions, behavioral representations, and deployment details already produced by steps 1 through 6 — there has to be something concrete to refactor and evaluate alternatives against. But the process is described as iterative, not linear, because refactoring and evaluating alternatives at step 7 routinely surfaces problems or better approaches that require revisiting earlier steps — for example, evaluating an alternative might reveal that a particular problem-domain design class (from step 1) should actually be restructured, or that the behavioral representation (from step 5) doesn't handle a state transition correctly. So while the steps are numbered to reflect a logical dependency order (you need classes before you can refine their behavior, you need a design before you can refactor it), the numbering describes a typical *first pass*, not a one-way pipeline — the loop back from step 7 to earlier steps is an explicit, expected part of the process, matching the "Iterative Design" principle from the design modeling principles list.

Q8. Scenario: A team packages all of their "utility" classes — a date formatter, a string helper, a logging wrapper, and an email-validation class — into one shared package because "they're all small helper classes." Two months later, a project that only needs

the date formatter is forced to pull in, test against, and redeploy the entire utility package whenever the unrelated email-validation logic changes. Which packaging principle was violated, and what should the team have done differently?

Answer: This violates the **Common Reuse Principle (CRP)** — the classes were grouped by superficial similarity ("they're all small helper classes") rather than by whether they're actually reused *together*, and in this scenario, the date formatter and the email validator clearly aren't reused together by the same consumers, since one project needs only the date formatter. Because CRP was violated, every consumer that depends on the shared package is forced to depend on classes it doesn't actually use, meaning changes to any one of those unrelated classes (the email validator, in this case) ripple out to consumers who never needed it at all — exactly the unnecessary recompilation/retesting/redeployment cost CRP exists to prevent. The fix is to split the "utility" package along actual reuse boundaries rather than superficial category — for example, a package containing just date/time-related classes, and a separate package for validation-related classes — so that a consumer needing only date formatting depends on a package containing only date-related classes, and is never affected by changes to code it has no relationship to.